

xspend, Engineering Reference

Last updated: July 2026

This is the orientation doc for anyone working on the backend. It covers what the product is, what runs where, how data flows, the core logic, the thresholds we chose and why, the bugs that have already bitten us, and the areas that are still fragile.

Read the "Landmines" and "Bug history" sections before changing anything in the upload path. Most of the hard-won knowledge in this codebase is there.

1. What xspend is

A personal finance analytics web app. Users get their transactions in, and the app categorizes them, deduplicates them, and turns them into a dashboard, insights, and projects.

Transactions arrive by two paths:

1. **Manual statement upload.** The user uploads a PDF, CSV, XLSX, or OFX file and we parse it ourselves.
2. **Bank connection via Plaid.** (Planned, not built yet.) Plaid handles extraction and returns clean, structured transaction data.

Manual upload is a deliberate product decision, not a fallback. A meaningful share of the US market banks with credit unions and small institutions, and many of those users will not connect their bank credentials to a third party. Most competing apps are Plaid-only, which excludes exactly those people. Manual upload is how we serve them.

Architectural consequence: the two paths have **isolated ingestion** and a **shared core**. Different on-ramps, same highway.

Layer	Manual	Plaid (planned)	Shared?
Ingestion	PDF/CSV parse	Plaid API + webhooks	Isolated
Normalization	parser output	Plaid mapper	Separate, but output shape is identical
Dedup			Shared
Classification			Shared
Storage			Shared (one table)
Dashboard / insights			Shared

This matters more than it looks. A user may both connect Plaid *and* upload a statement covering the same month. The same Starbucks charge then arrives twice, once as Plaid's clean `"Starbucks"` and once as our PDF's `"Debit Card Purchase Starbucks 8007827282"`, with no shared ID. **Only the fingerprint's merchant canonicalization can match those.** That is why the dedup hardening has to be solid before Plaid is connected, not after.

2. Stack and tools

Concern	Tool	Notes
Frontend	React + Vite + Tailwind	

Concern	Tool	Notes
Frontend hosting	Vercel	<code>xspend.vercel.app</code> , auto-deploys on push to <code>main</code>
Backend	FastAPI + SQLAlchemy	
Backend hosting	Render (Pro tier)	<code>xspend-io.onrender.com</code> , auto-deploys on push to <code>main</code>
Database (prod)	Neon / Postgres	Branch-based. Always confirm you are on the production branch before running SQL.
Database (local)	SQLite	<code>backend/financeai.db</code>
PDF text extraction	<code>pdfplumber</code>	
LLM parsing fallback	Anthropic API , Claude Haiku 4.5	model string <code>claude-haiku-4-5-20251001</code>
Source control	GitHub	<code>github.com/xspend/xspend.io</code> (private)
CI	None yet	See open issues. CD exists via Render/Vercel auto-deploy.

Render was upgraded from free to Pro to eliminate cold starts and spin-down. On the free tier the service slept after ~15 minutes idle, and the first request after a sleep caused stale-connection SSL crashes against Neon. If you see anything resembling that class of error again, check the tier first.

There is no CI. Every push to `main` deploys straight to production for real users. This is a known gap and is on the roadmap.

3. Repo layout

Local working directory: `~/Desktop/financeai` (`backend/` + `frontend/`).

Backend, core files

File	Owns
<code>main.py</code>	FastAPI app, all endpoints, the upload flow, the dedup loop
<code>parser.py</code>	File-type routing, bank detection, template parsing for known banks, routes failures to the LLM fallback
<code>llm_fallback.py</code>	LLM extraction for banks we have no template for. Text extraction, PII stripping, chunking, reconciliation
<code>classifier.py</code>	Fingerprint generation and merchant canonicalization
<code>fixed_classifier.py</code>	Fixed-vs-variable classification, recurrence detection, the classification index
<code>ai_chat.py</code>	The five templated insight prompts (no LLM, all deterministic)
<code>insights.py</code>	Dashboard insight calculations
<code>credit_engine.py</code>	Credit-card statement credits and rewards. Distinguishes real refunds from ineligible ones (cashback, welcome bonuses, points redemptions), maps Amex program credits to categories, computes net category spend after credits

File	Owns
<code>models.py</code>	SQLAlchemy models
<code>database.py</code>	DB connection and session setup
<code>auth.py</code>	Authentication
<code>migrate.py</code>	DB migrations
<code>pdf_parser_xy.py</code>	Coordinate-based PDF parsing helpers

Backend, not core

`backend/` also contains roughly fifty `patch_*.py` / `patchXXX_*.py` scripts. **These are spent, one-time migration scripts that have already been applied and committed.** They are historical noise. They should be archived to `_archive/applied_patches/` so the live codebase is legible. Do not treat them as live code.

Also present: `isolation_audit.py`, `isolation_audit_v2.py` (ad-hoc audit scripts), `test_fingerprint.py`, `test_slice2_local.py` (ad-hoc test scripts, not a real suite).

Frontend

`frontend/src/`: `main.jsx`, `App.jsx`, `lib/config.js`, and `pages/` containing `Landing`, `Login`, `Signup`, `Onboarding`, `Waitlist`, `Dashboard`, `Upload`, `Transactions`, `Chat`, `Goals` (projects), `Settings`, plus `Navbar`, `Sidebar`, `AppHeader`.

4. Data flow

The spine of the system. Everything else hangs off this.

```

upload (POST /upload)
|
v
parser.parse_statement()
|-- detect file type (csv / xlsx / pdf / ofx)
|-- detect bank from text
|-- template parse
|   |-- success -> rows
|   |-- returns empty      --\
|   |-- bank is Unknown    ---> LLM FALLBACK
|   |-- raises an exception --/
|
v
llm_fallback.parse_statement() (only when the template can't read it)
|-- pdfplumber text extraction
|-- strip PII
|-- single LLM call
|   |-- response truncated? -> boundary-aware chunking, extract each,
merge, dedup overlap
|-- reconcile against the statement's own opening/closing balances
|
v
main.py upload loop
|-- for each row: compute fingerprint (classifier.generate_fingerprint)
|-- DEDUP: (a) FITID -> (b) exact fingerprint -> (c) fuzzy
|-- save survivors
|
v
fixed_classifier.classify_all_transactions()
|-- build classification index ONCE
|-- classify only the newly-saved rows (bulk update)
|
v

```

Postgres (Neon)

|

v

dashboard / insights / chat / projects

5. Core logic

5.1 Parsing and the LLM fallback

`parse_statement()` routes by file type, tries a template parse, and falls back to the LLM.

The fallback fires in three cases:

1. The template parse returns **no rows**.
2. The bank is **unrecognized** (`Unknown Bank`).
3. The template parser **raises an exception**.

Case 3 was added late and matters. See the PNC bug in section 8.

`llm_fallback.parse_statement()` extracts text with pdfplumber, strips PII, and makes one LLM call. If the response comes back truncated (`stop_reason == "max_tokens"`) or fails to parse as JSON, it retries with **boundary-aware chunking**:

- Split into chunks of ~55 lines, but only cut **between transactions**, detected by a date at the start of a line (`__DATE_AT_START` matches `06/15`, `Jun 15`, `2026-06-15`).
- **Overlap 6 lines** across each cut, so a transaction sitting on a boundary is captured whole by at least one chunk.
- Merge, then **dedup the overlap** with `__dedup_key` (date + amount + normalized merchant stem, mirroring the app's fingerprint).
- Metadata: bank/account/period/opening from the first chunk that has them, closing from the last.

Reconciliation is the guarantee. The merged transactions are summed and checked against the statement's own printed opening and closing balances. If it doesn't close, the file is flagged (`needs_review`) rather than silently trusted. This is what makes chunked LLM output safe to rely on.

5.2 Fingerprinting (`classifier.py`)

This is the heart of dedup correctness.

```
if ext_id: fingerprint = sha256(bank | account | ext_id)
else:      fingerprint = sha256(bank | account | date | amount |
merchant_stem)
```

Merchant canonicalization is a three-step cascade (`_fingerprint_merchant`):

1. **Normalize.** Lowercase, strip processor prefixes (`sq *`, `tst*`, `dd *`), strip bank/parser transaction-type prefixes (`_FP_PREFIXES`: "debit card purchase", "pos purchase", "web pmt", "direct payment", "ach", etc., looped until stable), drop digit runs of 4+, years, punctuation.
2. **Alias match.** A "contains" match against `_MERCHANT_ALIASES`, roughly ninety national brands, returning a canonical name. Plus a `_stub` map for truncated tokens (`targ` -> `target`, `amzn` -> `amazon`, `sbux` -> `starbucks`, `wmt` -> `walmart`).
3. **Stem fallback.** First two significant tokens, for the long tail.

Why this exists: LLM parsing is non-deterministic. The same Starbucks charge comes back as `"Debit Card Purchase Starbucks"` on one parse and `"STARBUCKS 8007827282"` on the next. Without canonicalization those produce different fingerprints, dedup misses them, and a re-upload duplicates the user's entire statement. This is not theoretical; see section 8.

5.3 Dedup (`main.py` upload loop)

Three checks, in order. First match wins and the row is skipped.

Stage	Rule
(a) FITID	Incoming <code>(external_transaction_id)</code> (len > 5) already in the user's set
(b) Fingerprint	Exact fingerprint match against existing
(c) Fuzzy	Same account + same amount + same raw description, within ± 3 days

Rows surviving all three are saved. `(existing_fps)` is updated in-loop as rows are accepted; `(fuzzy_index)` is not (that asymmetry is a known bug, section 9).

5.4 Classification (`(fixed_classifier.py)`)

Decides fixed vs variable. Order of precedence:

1. **Keyword override.** Known recurring-subscription merchants are fixed even on a first upload, with no multi-month history needed.
2. **Category override.** Rent/mortgage, loan payments, insurance, subscriptions, bills are definitionally fixed regardless of amount. Without this, a large mortgage payment gets flagged as an amount outlier and wrongly marked discretionary.
3. **Recurrence signal.** Months present vs total months, plus amount variance.

The classification index (`(build_classification_index)`) is built once per batch:

`{merchant_key8 -> [transactions]}` plus the set of months present.

`(recurrence_signal)` looks up its merchant in $O(1)$ instead of rescanning every transaction. See section 8 for why this was necessary.

5.5 Money logic (the rules everything downstream depends on)

Net cash flow:

- **Money In** = income + refunds + zelle-in
- **Money Out** = net_expenses (gross expenses – card credits) + cash withdrawn + zelle-out
- **Credit card payments are excluded entirely.** Counting a payment from checking to a credit card as spending double-counts the original purchases.

Sign convention: negative = money out, positive = money in. Amex needed a three-layer fix to hold this (section 8).

Card statement credits are contra-expenses, subtracted from category spend rather than counted as income. `credit_engine.get_net_category_spend()` owns this.

Discretionary vs fixed: `_FIXED_CATS` (Rent/Mortgage, Loan Payment, Insurance, Bills & Utilities, Credit Card Payment, Education, Government & Taxes) are excluded from "spending pace" style metrics. A mortgage is not spending pace.

5.6 Insight prompts (`ai_chat.py`)

Five prompts, **all templated**. No LLM call, free, instant, deterministic. Capped per user per month via the `chat_log` table. Free-text chat is built (`get_ai_response`) but intentionally disabled for beta on cost grounds.

1. `net_cash_flow`: headline + money in/out composition
2. `purchase_affordability`: where to free up money, trims capped at ~1/3 of any category's occurrences so suggestions stay gentle
3. `lifestyle_creep`: where spending is drifting up (discretionary only)
4. `subscription_scan`: real subscriptions only (once/month + amount stable within 8%), price rises within those, same-day duplicate charges
5. `spending_velocity`: daily average, heaviest day, first vs second half. Discretionary only.

Every prompt carries an honest disclaimer about the data behind it (months of history, gaps, card credits offsetting).

6. Thresholds and magic numbers

Everything tunable, in one place. Most of these were chosen empirically against real statements.

Value	Where	Meaning
<code>0.90 / 0.80 / 0.75 / 0.65</code>	<code>parser.check_parse_threshold</code>	Valid-row ratio required to accept a parse, by mode (template / generic_table / generic_text / ocr)
<code>MAX_TOKENS = 8000</code>	<code>llm_fallback</code>	Near Haiku's output ceiling. Large statements exceed it, which is why chunking exists
<code>TOKEN_CEILING</code>	<code>llm_fallback.parse_statement</code>	Cost guardrail. Files above this are rejected rather than sent to the LLM
<code>target = 55 lines</code>	<code>llm_fallback._split_boundary_aware</code>	Chunk size
<code>overlap = 6 lines</code>	<code>llm_fallback._split_boundary_aware</code>	Lines repeated across each cut so boundary transactions survive
<code>±3 days</code>	<code>main.py</code> dedup	Fuzzy match window
<code>len(ext_id) > 5</code>	<code>main.py</code> dedup	Minimum length to trust an external id

Value	Where	Meaning
<code>first 2 tokens</code> , <code>[:32]</code>	<code>classifier._fingerprint_merchant</code>	Merchant stem length
<code>[:8]</code>	<code>fixed_classifier</code>	Merchant key prefix used for rules matching and the classification index
<code>8%</code>	<code>ai_chat.prompt_subscription_scan</code>	Max amount variance for a charge to count as a real subscription
<code>>5% and >= \$1</code>	<code>ai_chat.prompt_subscription_scan</code>	Threshold for reporting a price rise
<code>>= \$5</code>	<code>ai_chat.prompt_subscription_scan</code>	Minimum amount for a same-day duplicate-charge flag
<code>count // 3</code>	<code>ai_chat.prompt_purchase_affordability</code>	Max share of a category's occurrences we will suggest trimming
<code>1.15x</code>	<code>ai_chat.prompt_spending_velocity</code>	Ratio at which first vs second half is called uneven

7. Landmines

Read this section twice.

Local backend points at production by default

`backend/.env` sets `DATABASE_URL` to **Neon**. A local `uvicorn` will therefore read and write **production data** unless you override it:

```
bash
DATABASE_URL="sqlite:///./financeai.db" uvicorn main:app --port 8000
```

This is one stray `DELETE` away from real damage. Fix the default or export the override every time.

zsh eats `!` in strings

Writing test snippets inline with `python3 -c "..."` breaks on `!` (history expansion), including inside f-strings. Write a `.py` file instead.

Silent excepts

Every serious bug in this codebase so far was hidden behind an exception handler that swallowed the real error, or behind a missing log line. See section 8. **When something fails inexplicably, add loud logging before theorizing.** It has cracked every one of these in a single upload.

LLM parsing is non-deterministic

The same statement parsed twice produces slightly different descriptions. Any logic that assumes stable text will break. This is why fingerprints canonicalize the merchant.

Uploads are synchronous

Parse, dedup, and classify all run inside the HTTP request. Large statements (especially chunked LLM ones) take 45-60s and the frontend times out **even though the backend finishes and the data saves correctly**. Users see a false failure and re-upload, which is how one tester accumulated 342 rows for a single month. Tell testers their data saved. The real fix is async upload with polling.

8. Bug history

These are the bugs that have already cost real time. They are worth knowing because several were subtle, and their root causes are still shapes to watch for.

`max_id` collision (fixed)

New transaction IDs were derived from `COUNT(*)` rather than `MAX(id)`. After any deletion, `COUNT(*)` produced an ID that already existed, every insert in the batch collided, and the exception handler reported them as **duplicates**. Entire uploads vanished silently while the logs cheerfully said "skipped: duplicate". Fixed by using `MAX(id)`. **Lesson: an except that mislabels the error is worse than no except.**

Amex sign convention (fixed)

Charges and refunds had inverted signs. Required a three-layer fix:

`parser.fix_amount_for_bank` (use `-amount`, not `-abs(amount)`), `classifier` (a positive amount on a card is a refund), and food-keyword handling for `SQ*` / `TST*` merchants.

Cross-user data leak (fixed)

The fingerprint had a **globally unique** DB constraint. Two users with the same transaction collided, and one user's row was attributed to the other. Fingerprints must be unique per user, never globally.

PNC: silent template crash (fixed)

`parse_pdf_structured` returned `(None)` for PNC's layout. The caller did `(raw, detected_bank = parse_pdf_structured(...))`, which raised `(TypeError: cannot unpack non-iterable NoneType)`. That was caught by a broad `(except Exception)` that raised `(ValueError("Could not parse this file"))` **before** the LLM fallback block could run. So PNC died silently: no `(PARSED:)` line, no fallback log, no traceback, and a `(200 OK)` with `({"success": false})`.

Three theories and several days went into this before instrumentation found it in one upload. Fixed by logging the real exception and falling through to the fallback with `(raw = [])` instead of raising. **The underlying `(parse_pdf_structured)` returning `(None)` was never root-caused, only routed around.**

PNC: JSON truncation (fixed)

Once the fallback was reached, PNC's statement was large enough that the LLM's JSON response hit `(MAX_TOKENS = 8000)` and was cut off mid-string: `(Unterminated string starting at char 22238)`. Zero rows, and the error was cryptic because `(stop_reason)` was never checked. Fixed with the chunking described in 5.1.

PNC: 342 rows for one month (fixed, and the reason fingerprints changed)

After chunking worked, the tester had **342 distinct rows** for a single month, roughly triple reality. Cause: LLM non-determinism across the many debug re-uploads. Each attempt phrased descriptions slightly differently, so fingerprints differed, so dedup let them through as distinct. This is what forced the parse-stable fingerprint work in 5.2. **Any LLM-parsed bank is a re-upload-duplication risk without it.**

O(n²) classification (fixed)

`(classify_all_transactions)` called `(classify_transaction(tx, expense_txs))` for each of N transactions, and `(recurrence_signal)` scanned all N three times per call, each with a `(normalize_merchant())` call. That is $\sim 3N^2$ string normalizations: 15.46s for 329

rows, and minutes for a user with a few thousand. Fixed with `build_classification_index`: **152 transactions now classify in 0.11s**. Fix the algorithm, not the scope.

Others (fixed)

Apple Card and Amex bank detection failures. Insights filter bugs. Neon idle-connection SSL crashes (a symptom of Render's free tier spinning down; resolved by the Pro upgrade).

9. Known open issues

These are real, verified, and unfixed. Two independent engineering reviews of the dedup logic produced this list; each finding below has been confirmed against the code.

Isolation

#	Issue	Impact
1	<code>credit_engine.get_net_category_spend(db, period)</code> is not scoped by user . No <code>user_id</code> parameter. Flagged in a prior hardening pass and still open.	Cross-user data leak
2	The dedup routine takes plain <code>existing</code> / <code>incoming</code> lists with no user identifier . Correctness depends entirely on the caller never mixing two users' rows. Nothing in the function enforces it.	Cross-user merge if a caller ever gets scoping wrong
3	<code>bank</code> is a single string passed once per call. If a caller ever passes two banks in one batch, the same account name collides across them.	False merge

Dedup correctness

#	Issue	Impact
4	Two genuinely separate same-day, same-amount, same-merchant charges produce an identical fingerprint and the second is dropped. Also bites from the <code>(existing)</code> side: a user with two identical charges who re-uploads a statement containing those two plus a genuinely new third loses the third.	Loses a real transaction
5	A short <code>(ext_id)</code> (≤ 5 chars) is distrusted by the dedup fast-path but is still baked into the fingerprint hash by <code>(generate_fingerprint)</code> , which trusts any non-empty value. Two different charges sharing a short id collide.	Loses a real transaction
6	<code>(amount = None)</code> is coerced to <code>(0)</code> via <code>(t.get("amount", 0) or 0)</code> and saved as a real \$0.00 transaction.	Corrupts balances
7	Substring alias matching over-matches. "MyTarget Fitness Studio" contains "target" and collapses to the retailer.	False merge
8	FITID fast-path ignores <code>(account)</code> , unlike the fingerprint path which always includes it. Same id across two accounts wrongly merges.	False merge
9	<code>(account)</code> / <code>(bank)</code> are not case- or whitespace-normalized before hashing. "Checking" and "checking" produce different fingerprints.	Missed duplicate
10	<code>(fuzzy_index)</code> is built once from <code>(existing)</code> and never updated as incoming rows are accepted, unlike <code>(existing_fps)</code> which is. Two fuzzy-duplicate rows in the same batch both survive.	Missed duplicate
11	Fuzzy compares the raw description, not the normalized merchant. A re-upload where both the description and the settlement date drift (aggregators commonly do both) is caught by neither the fingerprint check	Missed duplicate

#	Issue	Impact
	(date differs) nor the fuzzy check (description differs). This directly violates our own "re-uploading produces zero new rows" requirement.	
12	Fuzzy ignores <code>(ext_id)</code> entirely. Two transactions the bank gave <i>different</i> ids still merge.	False merge
13	<code>(ext_id)</code> is stripped on the incoming side only , not on <code>(existing)</code> . A stray space on a saved row falls through both rule 1 and rule 2.	Missed duplicate
14	No validation layer. Rows with an empty date and/or description are saved as-is. Whether dedup or the parser should reject these is undecided.	Bad data

Note on #4: there is an inherent ambiguity the current data model cannot resolve. Nothing distinguishes "the parser emitted this line twice" from "the user really bought two identical \$20 items" without a statement line id or sequence number from the source. Occurrence counting (treat the Nth incoming match as a duplicate of the Nth existing occurrence, not the 1st) handles the common case, but this should be flagged as a modeling limit rather than something fully fixable with today's fields.

Infrastructure and product

#	Issue
15	No CI. Every push deploys straight to production. No automated tests gate anything.
16	No automated test suite. All verification to date has been manual terminal checks.
17	Uploads are synchronous and large statements time out on the frontend even though the backend finishes.
18	Temporary <code>[DEDUP]</code> and <code>[llm_fallback]</code> debug logging is still live in production.

Issue

-
- 19 ~50 spent `patch_*.py` scripts clutter `backend/`.
-
- 20 `parse_pdf_structured` returning `None` (the original PNC crash) was routed around, never root-caused.
-
- 21 The Anthropic API key should be rotated.
-

10. Where the product is going

Rough order. Detail lives in the roadmap.

1. **Auth and accounts.** Real user IDs, forgot-password, login email, account settings, account deletion (which must cascade across every user-owned table and touch nobody else's data, an isolation problem in its own right).
2. **Ingestion.** Parser hardening, async upload, then **Plaid**.
3. **Data integrity.** Everything in section 9.
4. **Notifications.** Opt-in, blueprint pending.
5. **Growth.** Dashboard and insights accuracy plus edge cases around how much data a user actually has, chat at 20 prompts/month with 5 pre-prompts, projects, and a what-if calculator.

One idea worth capturing: for the genuinely ambiguous dedup cases (#4), surfacing them in a review UI with a confidence score rather than forcing the code to guess. We already have `needs_review` / `review_status` on transactions and no UI for them.